Why a Model Zoo?

From a 100 GB Tarball to an Off-the-Shelf Product Experience

What PAVS needs from a Model Zoo — and what we must not turn it into

1 · The Off-the-Shelf Promise

What an application developer actually wants

- Download a model → **start building the application today**
- No knowledge of layers, kernels, quantization, or backends
- Just a **prepackaged model, best-suited to the task**, ready to run on AMD hardware

> The developer knows the **inputs and outputs** of the model and builds his product around them — nothing more.

The experience we are selling

Pick a task — detect / classify / segment / LLM

↓

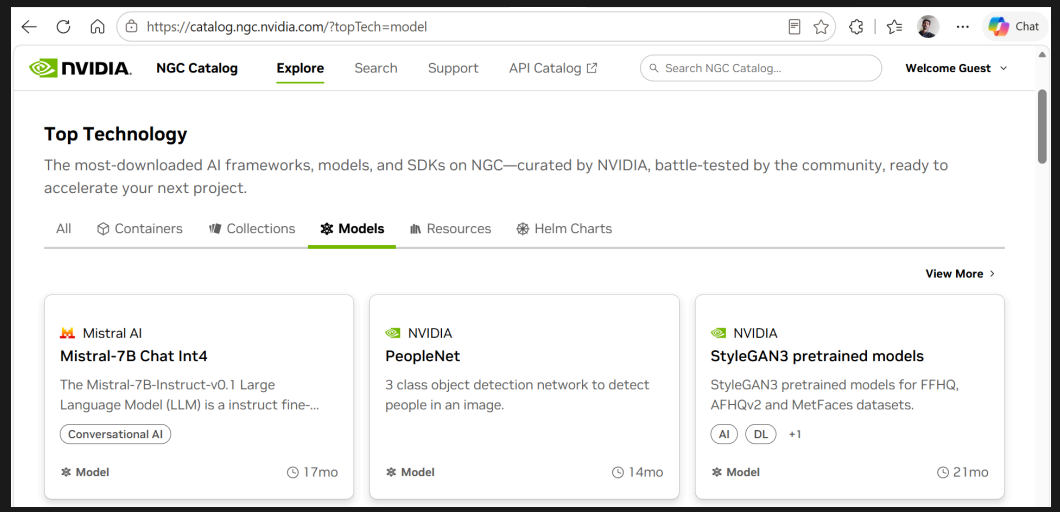
Download the packaged model — already optimized

↓

Build the product — runs on CPU · GPU · NPU

> A model zoo is what turns **"here are some weights"** into **"here is a product you can ship."**

2 · NVIDIA's Blueprint: NGC ↔ Isaac ROS



NGC Catalog — curated, prepackaged models (PeopleNet, Mistral-7B Int4, …)

The pattern that makes it work

- **NGC Catalog** hosts the ready-to-run models
- **Isaac ROS** consumes them directly into robotics pipelines
- The developer pulls a model and it *just runs* — no export, no tuning

AMD needs exactly this pairing:
a **zoo** that hosts optimized models + an **SDK / robotics stack** that consumes them off-the-shelf.

AMD Model Zoo hosts weights ↔ **PAI SDK / AARTS** consumes them

3 · The Tarball Problem

How we ship today

- **Every model** is packaged into the release tarball
- Add optimized variants of CNNs **and** LLMs and it balloons fast
- Realistically **100+ GB** — and growing with every new variant

Monolithic Tarball · 100+ GB

YOLOv12 · YOLO26 fp16/int8 · MobileSAM · CenterPoint · Llama-3 int4 · SmolVLA · + every variant

↓ forced onto every disk

User who only wanted **2 CNNs** still pays the full 100+ GB

Every user carries **the entire zoo on their disk** — even if they only ever use a couple of CNNs.

A zoo fixes this: **pull only the model you need**, keep the SDK lean.

4 · With the Zoo vs Without — Two Flows to the Same Model

AIG hands us a PyTorch model — the zoo stores every optimized artifact so the customer never has to

AIG PyTorch → ONNX · MIGraphX · Vitis-AI → Quantize · AWQ / INT4 → NPU flow → Zoo stores each stage ✓

✓ With the Zoo — one download

Pick the artifact you need

ONNX · MIGraphX · Vitis-AI · INT4 · NPU

↓

Download — pre-baked

any stage, on one go

↓

Build the application

minutes to first inference

Every optimized artifact is **ready to pull** — the heavy lifting was done once, by us.

✗ Without the Zoo — run every stage

PyTorch model

↓

Export to ONNX — MIGraphX / Vitis-AI

↓

Quantize — AWQ / INT4 + calibration

↓

NPU compile — kernel-level opt

↓

…only now usable

Every customer re-runs the **whole toolchain on their own disk** — hours of work, before line one of the app.

5 · Save the Optimization Stages — Don't Ship Them Raw

This is why NVIDIA & Qualcomm pre-bake artifacts — and why PAVS runs its own model zoo today

The zoo stores every stage as a downloadable artifact

- PyTorch → ONNX → MIGraphX / Vitis-AI graphs
- AWQ-quantized ONNX · INT4 + calibration data
- Kernel-level optimized NPU builds

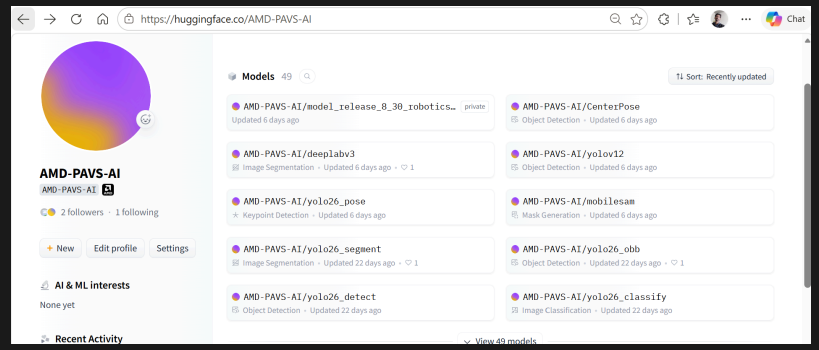
> **If we don't save these stages,** the end customer has to run PyTorch → ONNX → INT4 **on his own disk at usage time** — slow toolchains, calibration data, hours of work. **Poor user experience.**

This is why NVIDIA & Qualcomm pre-bake artifacts

- NVIDIA NGC — ready-to-run per target
- **Qualcomm AI Hub** — per-backend precompiled models

The current PAVS model zoo — live today

- Built **close to the Qualcomm model** — graphics not there yet, but the structure is live and already serving models



AMD-PAVS-AI on Hugging Face — 49 models, per-task, ready to pull

6 · Two Kinds of Users — Two Kinds of Products

AIG Git + Model Zoo serves…

The Research Engineer

- Knows the **nitty-gritty** of the model internals
- Rewires layers, swaps operators, debugs graphs
- Cares *which layer breaks*, *which workflow is GPU-heavy*, *which lighter variant to substitute*
- Works **inside** the model

Audience: people who build and modify models.

PAVS serves…

The Application Engineer

- Uses AI models as **black boxes**
- Knows only the **inputs and outputs**
- Does **not** need to know which layer breaks or which variant is lighter
- Builds the **application around** the model

Audience: people who ship products using models.

Our zoo brings a **different kind of user** onto AMD silicon — and needs to be shaped for **them**, not for the researcher.

7 · What PAVS Expects From a Model Zoo

Just a storage space — like Hugging Face

- We **don't care who owns or maintains it** — could be AMD central-wide infrastructure
- One requirement: it **hosts weights and serves them to customers**

What we do need: operational flexibility

- Freedom to **create directories** / structure as our workflows demand
- **Store the weights** for every optimization stage
- **Make them public** for customer consumption on our schedule

Model Zoo = Storage + Freedom

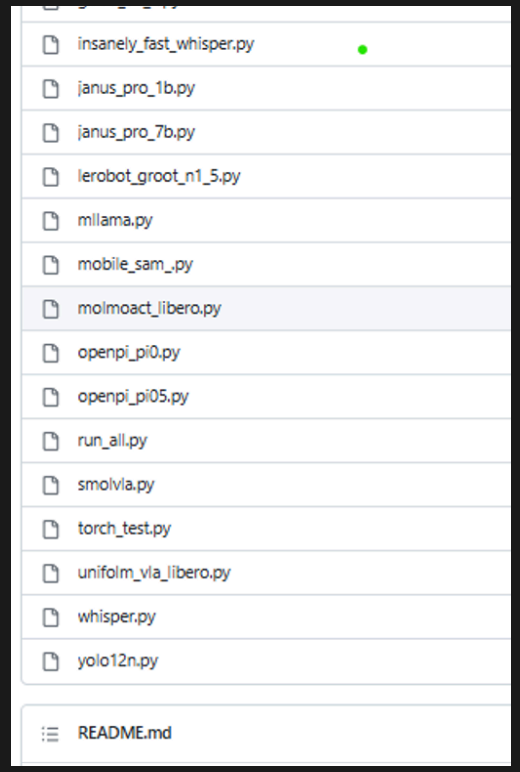
- 📁 Create our own directory structure
- 💾 Store weights for every stage
- 🌐 Publish for customers directly
- 🔒 Control what is public, and when

> Think **Hugging Face for AMD**: a backing store we operate freely — not a system we must conform to.

The Founding Promise — Research Desk → Vertical Software

1. PAVS transforms raw researcher code into production-ready vertical software

What we receive: Research Repo



Flat list of scripts — no structure, no device flows, no Make targets

What we receive: Research Code

```
# rocm-scripts/test/pytorch/yolo12n.py
#........
# .......
# .......
#........
# .......
# .......
recalled = gt_labels & detected_classes
recall = len(recalled) / len(gt_labels)
sanity_pass = recall >= RECALL_THRESHOLD
assert sanity_pass, (
  f"recall {recall:.0%} < {RECALL_THRESHOLD:.0%}")

return lats, {
  "architecture": "CNN",
  "task": "object_detection",
  "num_detections": num_detections,
  "recall": round(recall, 4),
}
if __name__ == "__main__":
  sys.exit(run(run_benchmark))
```

Single script, no structure, no device flows

What we deliver: Vertical Software

```
examples/yolov12/
├── Makefile
├── METRICS_TABLE.md
├── README.md
├── app/
│   ├── routes/
│   ├── services/
│   ├── main.py & config.py
├── scripts/
│   ├── benchmark.py
│   ├── export_onnx.py
│   ├── profile_model.py
│   ├── run_val.py
│   └── update_metrics_table.py
├── tests/
│   ├── test_api.py
│   ├── test_docker.py
│   ├── test_migraphx.py
│   └── test_yolo_workflow.py
├── weights/
├── datasets/
├── requirements.txt
├── requirements_cpu.txt
└── requirements_npu.txt
```

App, scripts, tests, per-device requirements — production-ready

Physical AI & Vertical Software: Researcher's Desk → Vertical Software — same commands, every device, production-ready.

AMD
together we advance_

8 · What We Should NOT Do

The initial promise: convert research code into vertical software — a zoo must not reverse that

The anti-pattern

If AIG **hosts** the zoo and mandates we place our models in it:

1. To appear on **their dashboard**, we must write research code that **fits their git repo**
2. We end up rebuilding a **research setup** — **not vertical software**
3. This turns our SDK into a mirror of **their repo and dashboard**

> This direction **nullifies the value-add** that Physical AI & Vertical Software brings — the exact opposite of our founding promise.

The right shape

- Our job (per the SW-stack promise) is **Research Desk → Vertical Software**
- Fitting AIG's repo/dashboard is **book-keeping**, not a **customer-facing** deliverable

Vertical software — customer-facing, product-grade

▲ keep on top

AIG repo / dashboard sync — do it as *book-keeping*, if at all

> **If it must be done, do it as book-keeping** — never let it turn the SDK into a customer-facing clone of the research repo.

The Ask

A model zoo that is storage + operational freedom — so PAVS ships an off-the-shelf, product-grade experience on AMD silicon

Host the optimized artifacts · keep the SDK lean · serve the application engineer · stay vertical software

AMD
together we advance_